

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 03 -

TESTES AUTOMATIZADOS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	7
MERCADO, CASES E TENDÊNCIAS	29
O QUE VOCÊ VIU NESTA AULA?	30
REFERÊNCIAS.....	31

EMSE

O QUE VEM POR AÍ?

É comum que códigos em notebooks falhem após algum tempo sem manutenção, especialmente se não adotamos testes automatizados. Esses testes garantem que o código funcione como esperado mesmo após alterações, prevenindo bugs e facilitando a reprodutibilidade. Em ciência de dados, testes unitários avaliam funções individuais e testes de integração analisam o funcionamento conjunto dos módulos. Automatizando esses testes, detectamos rapidamente regressões antes que impactem resultados.

Ferramentas como Pytest (Python) ou JUnit (Java) simplificam a criação desses testes. Práticas como Desenvolvimento Orientado por Testes (TDD) incentivam implementar os testes antes do código, promovendo modularidade e confiança nas mudanças. Assim, testes automatizados profissionalizam o desenvolvimento em ciência de dados, tornando o código mais confiável, fácil de manter e menos sujeito a erros.

HANDS ON

Vamos agora aplicar esses conceitos em um exemplo prático, usando Python e Pytest. Suponha que estamos desenvolvendo funções para um pipeline simples de ciência de dados: uma função que filtra valores inválidos e outra que calcula uma métrica agregada. Queremos assegurar que cada parte funcione isoladamente e que, integradas, produzam o resultado correto – adotando inclusive um pouco de TDD no processo.

Cenário: temos uma lista de números que pode conter valores negativos indesejados. Queremos primeiro remover os negativos e depois calcular a média dos restantes. Vamos desenvolver duas funções: `filtra_negativos(dados)` e `calcula_media(dados)`. Em seguida, escreveremos testes unitários para cada função e um teste de integração para o pipeline completo (as duas funções em sequência). Usaremos Pytest para criar esses testes.

Para seguir TDD, poderíamos começar escrevendo os testes antes mesmo das funções. Aqui, por brevidade, apresentaremos já as implementações juntamente com os testes correspondentes, mas mantendo o espírito: cada funcionalidade vem acompanhada de um teste que verifica seu comportamento esperado.

```
# Implementação das funções alvo do pipeline
def filtra_negativos(valores):
    """Retorna uma lista contendo apenas os valores não-negativos de 'valores'."""
    return [x for x in valores if x >= 0]

def calcula_media(valores):
    """Retorna a média aritmética dos valores da lista, ou 0 se a lista for vazia."""
    if not valores: # lista vazia
        return 0
    return sum(valores) / len(valores)
```

Código-fonte 1 – Implementação das funções alvo do pipeline em Python
Fonte: Elaborado pelo autor (2025)

Nas funções anteriores, `filtra_negativos` remove números negativos de uma lista e `calcula_media` calcula a média ou retorna 0 se a lista estiver vazia (evitando divisão por zero). Agora, criaremos os testes. No Pytest, um teste unitário é apenas uma função cujo nome começa com `test_`. Usaremos também a facilidade do `@pytest.mark.parametrize` para testar vários cenários em um único teste unitário, passando diferentes entradas e saídas esperadas.

```
# test_pipeline.py - arquivo de testes (simulação)
import pytest
from pipeline import filtra_negativos, calcula_media # assumindo que implementações estão em
pipeline.py

# Teste unitário para filtra_negativos
@pytest.mark.parametrize(
    "entrada, esperado",
    [
        ([-1, 5, 0, -3, 2], [5, 0, 2]), # mistura de negativos e não-negativos
        ([1, 2, 3], [1, 2, 3]), # nenhum negativo a remover
        ([-5, -6], []), # só negativos, resultado vazio
        ([], []), # lista vazia permanece vazia
    ]
)
def test_filtra_negativos(entrada, esperado):
    assert filtra_negativos(entrada) == esperado

# Teste unitário para calcula_media
@pytest.mark.parametrize(
    "entrada, esperado",
    [
        ([5, 5, 5], 5), # média de valores iguais
        ([2, 4], 3), # média inteira exata
        ([2, 3, 4], 3), # média não-inteira
        ([], 0), # lista vazia -> 0 (caso especial)
    ]
)
def test_calcula_media(entrada, esperado):
    assert calcula_media(entrada) == esperado

# Teste de integração do pipeline completo (filtragem seguida de média)
def test_pipeline_completo():
    dados_brutos = [-1, 2, 4, -3] # entrada contendo valores negativos e positivos
    filtrado = filtra_negativos(dados_brutos)
    resultado_final = calcula_media(filtrado)
    assert filtrado == [2, 4] # após filtrar, espero apenas [2, 4]
    assert resultado_final == 3 # média de [2,4] deve ser 3.0 (float) ou 3 (int no Python
    resulta float 3.0, comparável a 3)
```

Código-fonte 2 – Exemplo de código para simulação de testes em Python

Fonte: Elaborado pelo autor (2025)

Como funciona? Nos testes mostrados, cobrimos múltiplos cenários para cada função: por exemplo, `test_filtra_negativos` verifica desde o caso básico até listas totalmente negativas ou vazias. Usamos `pytest.mark.parametrize` para definir essas entradas e saídas esperadas para cada cenário de forma concisa, em vez de escrever várias funções de teste repetitivas. Cada cenário é executado separadamente pelo Pytest. Já `test_pipeline_completo` é um teste de integração que executa as duas funções em sequência – simulando o pipeline inteiro – e confere tanto o resultado intermediário (lista filtrada) quanto o resultado final (média).

Rodando o Pytest, todos os testes devem passar. Se introduzirmos um erro, por exemplo, se `calcula_media` retornasse `sum(valores)` em vez de dividir pelo `length`, um dos testes falharia indicando a discrepância esperada vs. obtida. Esse

feedback rápido permite corrigir antes que o erro chegue em produção. Vale ressaltar que, na prática, separaríamos os testes em um arquivo `test_pipeline.py` (como simulado) e o código em outro (por exemplo `pipeline.py`). Ao rodar `pytest` na linha de comando, o framework descobriria automaticamente as funções `test_...` e executaria todas, reportando quais passaram ou falharam.

Além disso, poderíamos seguir o ciclo de TDD estritamente: escrever primeiro `test_filtro_negativos` (que falharia, pois a função não existia) e então implementar `filtro_negativos` até o teste passar; depois o mesmo para `calcula_media`, e assim por diante. Esse estilo garante que cada funcionalidade é pensada para ser testável desde o início e só é considerada "pronta" quando seu teste está verde. No fim, teríamos uma suíte robusta – se alguém no futuro modificar, por exemplo, `filtro_negativos` para um critério diferente, os testes acusariam qualquer efeito indesejado no comportamento original.

Observação: embora usemos Python aqui (justificando sua escolha por ser uma das linguagens mais utilizadas em ciência de dados), os conceitos se aplicam a outras linguagens. Em Java, por exemplo, escreveríamos métodos de teste semelhantes usando JUnit, com asserções como `assertEquals(expected, actual)` no lugar do `assert` do Python. O importante é adotar uma cultura de testes no desenvolvimento de projetos de ciência de dados, independentemente da tecnologia.

SAIBA MAIS

Nesta seção teórica, vamos nos aprofundar em conceitos e melhores práticas de engenharia de software relativas a testes automatizados, sempre contextualizando para projetos de Ciência de Dados. Abordaremos os diferentes níveis de teste e seu papel (unitário, integração e end-to-end), a técnica de Test-Driven Development (TDD) de forma mais detalhada e consideraremos aspectos de performance, recursos (CPU/GPU/memória) e desafios específicos de testar código de data science.

Também discutiremos como casos de uso matemáticos e estatísticos podem ser incorporados nos testes (por exemplo, testes baseados em propriedades e verificação de invariantes) e como ferramentas especializadas ajudam a manter a qualidade em pipelines de dados. Tudo isso mantendo uma linguagem acessível e referências a fontes confiáveis.

Tipos de teste: Unitário, Integração e End-to-End

No desenvolvimento de software, os testes são frequentemente categorizados pelo escopo que abrangem. Em ciência de dados, costumamos nos concentrar principalmente em testes unitários e de integração, mas vale entender também o papel de testes de ponta a ponta (E2E) e outros níveis para ter uma visão completa da pirâmide de testes.

Testes unitários: como visto, testa-se unidades individuais de código em isolamento. Isso significa verificar se funções ou métodos específicos produzem os resultados corretos para entradas controladas. Por exemplo, testar uma função de limpeza de dados para diferentes dataframes de teste e conferir se o output está correto e livre de anomalias esperadas.

Unit tests geralmente são white-box (caixa-branca) – o(a) desenvolvedor(a) conhece a implementação interna – e devem cobrir tanto o caminho feliz (inputs válidos esperados) quanto casos de borda (e.g. lista vazia, valores nulos etc.). Eles são escritos e executados pelo próprio indivíduo desenvolvedor durante a construção do código.

Em projetos de DS, testes unitários podem validar desde funções matemáticas simples (como cálculo de uma métrica estatística) até componentes de ML (por exemplo, verificar que uma função de normalização de dados está realmente mapeando os valores para a faixa esperada 0–1). Por serem focados e isolados, tendem a ser rápidos de rodar e baratos em termos de recursos, permitindo executá-los com frequência (até em cada commit de código).

Testes de integração: avaliam o comportamento de módulos combinados, ou seja, se as interfaces entre componentes estão corretas. Depois que funções individuais já passaram em seus unit tests, parte-se para verificar cenários onde elas interagem. Em ciência de dados, isso pode significar, por exemplo, testar uma pipeline completa: carregar dados -> transformar -> rodar modelo -> salvar resultados.

Um teste de integração forneceria um conjunto de dados de exemplo (talvez pequeno) e verificaria se o resultado final do pipeline está certo. Aqui não nos importamos apenas com cada etapa isolada, mas sim com o encadeamento: falhas de integração podem incluir um formato inesperado de dados entre etapas, suposições incorretas (ex.: um algoritmo espera dados ordenados, outro componente não os ordena) ou problemas de compatibilidade de tipos.

Diferentemente dos unitários, os testes de integração são frequentemente black-box (caixa-preta) em relação aos componentes – apenas checamos entradas e saídas combinadas – e podem envolver dependências externas (como um banco de dados em memória ou chamadas a serviços fictícios mockados). Em DS, um exemplo seria integrar um módulo de limpeza de texto com um de análise de sentimento e ver se, juntos, produzem a conclusão esperada para um texto de teste.

Esses testes tendem a ser mais lentos e custosos (pois exercem mais partes do sistema e podem usar mais dados) do que os unitários, então geralmente rodam com menos frequência (por exemplo, em cada integração contínua ou antes de uma release). Ainda assim, são cruciais para evitar a síndrome do “funciona na minha máquina” – garantem que componentes funcionem em conjunto como previsto.

Testes end-to-end (E2E): são testes que percorrem todo o sistema do início ao fim, em um ambiente o mais próximo possível do real. Em aplicações web, um teste E2E simularia um usuário navegando na interface até o banco de dados e de

volta. Em ciência de dados, um E2E poderia ser: em uma pipeline de dados, iniciar com dados brutos em um sistema de armazenamento, executar agendamentos ou fluxos (por exemplo, uma DAG no Airflow), gerar modelos e avaliar outputs finais, talvez até validando se um dashboard ou API retorna valores esperados depois de todo o processamento.

Essencialmente, verifica-se que todos os componentes, incluindo aqueles fora do nosso código direto, estão funcionando em conjunto – por exemplo, que um script criado roda sem problemas no ambiente Docker em produção, com as credenciais e configurações corretas. Muitas vezes, testes E2E em DS assumem a forma de reprocessar um pequeno lote real de dados através do pipeline completo em um ambiente de staging e conferir se as saídas batem com resultados previamente validados ou invariantes conhecidas.

Por exemplo, poderíamos rodar uma pipeline de treinamento de modelo com um dataset reduzido e checar se a métrica de acurácia no final atinge pelo menos um valor mínimo esperado. Esses testes fornecem alta confiança, porém são os mais lentos e complexos (podendo levar minutos ou horas e envolver recursos significativos, como GPUs para retreinar um modelo inteiro). Devem ser usados estrategicamente – como uma verificação final antes de colocar um modelo em produção ou antes de uma grande atualização na pipeline.

Uma boa prática é não depender só de E2E para qualidade (pois são poucos e demorados), mas usá-los como complemento de uma base sólida de testes unitários e de integração (a clássica “Pirâmide de Testes” sugere muitos testes unitários rápidos na base, alguns de integração no meio e poucos E2E no topo).

Além desses, há outros níveis e tipos de testes que podem ser relevantes conforme o contexto:

Testes de regressão: não são um nível distinto, mas um propósito – assegurar que novas alterações não reintroduziram erros antigos. Basicamente, toda a suíte de testes automatizados serve como teste de regressão. Ferramentas de CI/CD (Integração Contínua/Entrega Contínua) ajudam a rodar a suíte a cada mudança no código, garantindo regressão zero. O caso clássico de falha de regressão em finanças, o bug da Knight Capital, poderia ter sido evitado se

houvesse testes cobrindo o cenário de código legado não utilizado – veremos mais na seção de cases.

Testes de desempenho (performance): avaliam requisitos não funcionais, como tempo de execução e uso de memória sob carga. Em DS, podemos criar testes que garantem que um determinado processamento roda dentro de X segundos em um dataset de Y registros, ou que o uso de memória permanece abaixo de um limite durante a execução (detecção de vazamentos de memória). Embora não sejam testes de “correção lógica”, são importantes para pipelines de larga escala. Ferramentas como `pytest-benchmark` permitem medir tempos e comparar com bases históricas. Já frameworks de stress test podem simular volumes maiores – por exemplo, repetir mil vezes uma função de agregação sobre dados para avaliar escalabilidade.

Testes de schema ou qualidade de dados: especialmente relevantes em DS, focam não no código, mas nos dados que alimentam o código. Erroneamente às vezes ignorados na categoria de testes automatizados, ferramentas como Great Expectations e Deequ permitem definir expectativas sobre os dados (por exemplo: “a coluna Id não deve ter valores nulos nem duplicados” ou “a distribuição de valores de idade deve estar dentro de um range plausível”) e validar automaticamente cada novo lote de dados.

Embora não testem o comportamento do software em si, evitam que mudanças silenciosas nos dados quebrem suposições do pipeline. Podemos integrá-los em pipelines para rodar antes ou depois das transformações principais, acionando alertas se os dados violarem as regras esperadas.

Exemplo: a Calm (uma empresa de app de meditação) integrou verificações do Great Expectations em seu fluxo de ETL dentro do Airflow – assim a equipe de dados recebe alertas no Slack se, por exemplo, uma tabela recebida tiver menos linhas que o esperado, podendo parar a execução antes que dados incorretos propaguem. Esses testes de dados complementam os testes de código tradicionais, formando juntos um robusto controle de qualidade de ponta a ponta no projeto de DS.

Em projetos de ciência de dados, recomenda-se garantir cobertura suficiente nos níveis unitário e integração, conforme indicado no enunciado da aula.

“Cobertura” refere-se à porcentagem de código (ou cenários) exercitada pelos testes. Embora 100% de cobertura não garanta ausência de bugs, uma boa cobertura – principalmente das partes críticas – aumenta a chance de pegar problemas. Deve-se focar em cobrir:

- Funcionalidades centrais de cálculo e transformações (p. ex. funções de limpeza, engenharia de atributos, cálculos de métricas) com unit tests abrangentes.
- As principais interações entre módulos (p. ex. ingestão -> transformação -> output; treinamento -> serialização do modelo -> predição) com testes de integração.
- Se possível, um ou dois fluxos E2E representativos (p. ex. rodar todo pipeline em ambiente de teste com um pequeno sample de dados real) para validar integrações externas e configurações.

Em suma, usar múltiplos níveis de teste de forma adequada confere um feedback rápido em várias etapas: um erro de lógica simples gera falha imediata em um teste unitário (mais fácil de debugar, pois aponta exatamente a função problema), enquanto um erro de integração aparece em um teste de integração ou e2e indicando que “o conjunto não está fechando”.

Essa malha de segurança nos permite evoluir um projeto de ciência de dados com confiança, seja refatorando código para otimização, seja adicionando novos passos na pipeline, sem reintroduzir bugs já solucionados – se algo regressar, um teste falhará e nos alertará.

TDD (Test-Driven Development) e sua aplicação

Desenvolvimento Orientado por Testes (TDD) é uma técnica de desenvolvimento de software em que você escreve o teste antes da implementação do código. Vamos detalhar como funciona e discutir considerações de seu uso em ciência de dados.

No fluxo tradicional de desenvolvimento, implementamos uma função para depois criar testes que a verifiquem. No TDD, invertemos essa ordem: primeiro imaginamos como a função deveria se comportar e codificamos esse

comportamento esperado em um teste automatizado; somente então desenvolvemos a função real até que o teste passe. O processo básico segue três passos, frequentemente chamados de ciclo Red/Green/Refactor (NELSON, 2024):

1. **Red:** escreva um teste novo para alguma funcionalidade desejada. Execute os testes e veja o novo teste falhar (vermelho), já que a funcionalidade ainda não existe ou não está completa.
2. **Green:** implemente o código mínimo necessário para fazer o teste passar (verde). Aqui a prioridade é satisfazer a especificação do teste rapidamente, não escrever a solução perfeita – pode ser um código “gambiarra”, desde que passe.
3. **Refactor:** com o teste verde garantindo que a funcionalidade está correta, refatore o código – melhore a estrutura, elimine duplicações, generalize soluções –, mantendo os testes passando. Os testes atuam como rede de segurança para a refatoração, assegurando que o comportamento não mudou enquanto você limpa o código.

Depois disso, repete-se o ciclo para a próxima funcionalidade. O foco do TDD é que os testes descrevem o que o código deve fazer, servindo como especificação. Ao escrever o teste primeiro, você clarifica o requisito: “Dada entrada X, espero saída Y”. Essa clareza muitas vezes destaca detalhes que poderiam ser esquecidos se partíssemos direto para a implementação.

Além disso, como você quer que o teste seja simples de escrever e ler, tende a projetar interfaces (funções, classes) mais claras e modulares. Por exemplo, se para testar uma etapa de um pipeline é difícil isolar a função, isso é um indicador de que talvez o código esteja muito acoplado; no TDD, você sentiria essa dor cedo e ajustaria o design (por ex. separando responsabilidades em funções menores) para facilitar o teste.

Um benefício significativo relatado do TDD é que ele encoraja designs mais simples e coesos. Como observado na literatura: “Testes no TDD especificam o comportamento esperado e forçam o desenvolvedor a quebrar problemas complexos em partes testáveis”. Isso geralmente resulta em funções menores, com menos dependências globais, justamente porque isso torna a implementação mais fácil de testar.

Em ciência de dados, isso pode contrabalancear a tendência de escrever muito código exploratório monolítico em notebooks; o TDD “puxa” o(a) cientista de dados para pensar em componentes reutilizáveis e testáveis (por exemplo, uma função para limpar outliers, outra para combinar datasets etc., cada uma com seu teste).

Entretanto, é importante notar limitações e adaptações do TDD no contexto de ciência de dados. Um ponto levantado por Catherine Nelson (autora de *Software Engineering for Data Scientists*) é que nem sempre sabemos de antemão quais funções precisaremos em um projeto de data science. Durante a fase exploratória, o caminho até a solução pode ser incerto, diferentemente do desenvolvimento de um sistema com requisitos bem definidos.

Assim, escrever testes antes nessa fase inicial pode não valer o esforço, já que o código está evoluindo rapidamente e podendo ser descartado. Nelson sugere que TDD e testes em geral se tornam mais úteis após a fase exploratória, quando começamos a consolidar o código que será reutilizado ou colocado em produção. Nesse momento, sim, vale travar o comportamento com testes e então refatorar e otimizar o código com segurança.

Outra adaptação prática do TDD em DS envolve testar por comportamento esperado sem conhecer exatamente o resultado numérico. Por exemplo, ao desenvolver um algoritmo de aprendizado, talvez não saibamos de antemão qual exata acurácia ele deve produzir em determinado conjunto (seria estranho escrever um teste “espera-se acurácia = 0.8457”).

Em vez disso, podemos escrever testes que verifiquem propriedades do resultado: por exemplo, que a função de treinamento diminui a perda no conjunto de treino após algumas épocas, ou que um classificador treinado em um conjunto mínimo consegue obter desempenho melhor que aleatório em dados conhecidos. Esses são testes baseados em expectativas mais abstratas, porém ainda úteis para guiar o desenvolvimento.

Um caso comum é testar componentes complexos com invariantes matemáticas: se implementamos uma função de normalização de vetor, podemos não saber a saída exata para cada input arbitrário, mas sabemos que a norma resultante deve ser 1. Assim, podemos escrever primeiro um teste que gera um vetor

aleatório e verifica `norm(normaliza(v)) == 1` (dentro de alguma tolerância) antes mesmo de implementar a função. Essa é uma forma de TDD usando propriedade matemática em vez de exemplo específico; frameworks de property-based testing como Hypothesis suportam gerando automaticamente vários casos para validar a propriedade.

Em resumo, o TDD no contexto de ciência de dados deve ser usado com discernimento. Durante explorações iniciais, talvez seja ágil prototipar sem escrever testes para cada experimento; mas conforme o projeto se encaminha para produção ou consolidação, adotar TDD para novas features pode diminuir muito a incidência de bugs e facilitar a manutenção. Mesmo que não se siga TDD estrito a todo momento, pensar como em TDD – isto é, pensar “como vou testar isso?” ao escrever código – já melhora a qualidade. E uma vez que se tenha um conjunto de testes sólidos, pode-se refatorar e otimizar código científico (que muitas vezes começa com baixa performance ou pouca organização) com muito mais tranquilidade, sabendo que os testes darão confiança de não ter quebrado nada.

Boas Práticas de Testes para Projetos de Ciência de Dados

Agora que conhecemos os tipos de testes e a filosofia do TDD, vamos examinar algumas melhores práticas gerais para escrever testes eficazes em projetos de ciência de dados. Muitas delas são compartilhadas com testes de software em geral, enquanto outras são especialmente relevantes quando lida-se com dados, aleatoriedade, algoritmos complexos e uso de recursos como CPU/GPU.

1. Siga o padrão Arrange-Act-Assert (AAA): organização clara dos testes facilita a leitura e manutenção. Estruture cada teste em três seções:

- **Arrange (Preparação):** configurar os dados de entrada e quaisquer pré-condições necessárias.
- **Act (Ação):** executar a função ou módulo a ser testado com os arranjos definidos.
- **Assert (Verificação):** checar que os resultados obtidos conferem com o esperado.

Por exemplo, para testar uma função `conta_vogais(texto)`, você poderia:

```
# Arrange
texto = "hello"
esperado = 2
# Act
resultado = conta_vogais(texto)
# Assert
assert resultado == esperado
```

Código-fonte 3 – Exemplo de código em Python
Fonte: Elaborado pelo autor (2025)

Simples assim. Esse padrão evita testes desorganizados e deixa explícito o que está sendo testado e como. Em Pytest, muitas vezes usamos variáveis dentro do próprio teste para Arrange/Act e depois assert direto, enquanto no unittest usaríamos métodos como `self.assertEqual`. O importante é a clareza.

2. Escreva testes determinísticos (reprodutíveis): data science frequentemente envolve aleatoriedade – embaralhamento de dados, inicialização de pesos de modelos, separação de holdouts etc. Testes devem evitar falsos negativos (falhas intermitentes) causados por sorteio aleatório. Para isso:

- Fixe sementes de gerador de números aleatórios dentro dos testes, quando aplicável (por exemplo, ao testar um algoritmo K-means que inicia centroides aleatoriamente, faça `random.seed(42)` ou configure a biblioteca para usar seed fixa). Assim, o teste rodará o mesmo caminho toda vez.
- Se não puder fixar o random (por exemplo, ao testar uma função que deve produzir resultados estocásticos), então teste propriedades estatísticas. Exemplo: suponha que temos uma função `amostra_distribuicao(probabilidades)` que devolve um rótulo conforme distribuição de probabilidade. Não dá para prever o rótulo exato (pois é amostragem aleatória), mas podemos rodar, digamos, 10000 amostras em um teste e verificar que a frequência de cada rótulo está próxima (dentro de um intervalo de confiança) das probabilidades esperadas. Isso envolve estatística no teste – podemos calcular p-value de um teste qui-quadrado para ver se a distribuição difere significativamente do esperado e falhar o teste se for muito improvável (indicando erro na implementação da amostragem). Esse tipo de teste é mais complexo, mas útil para funções probabilísticas.

- Outra abordagem para aleatoriedade replicável é usar valores determinísticos no lugar de aleatórios durante testes. Por exemplo, se sua função faz `np.random.rand()`, você pode usar técnicas de monkeypatch ou injeção de dependência para substituir o gerador por um que retorna sempre a mesma sequência nos testes. Isso entra na esfera de mocking, discutida mais adiante.

3. Use fixtures e dados sintéticos pequenos para testes: fixtures são recursos fornecidos aos testes, por exemplo um pequeno dataframe de exemplo, ou um arquivo CSV de teste. O Pytest permite definir fixtures reutilizáveis facilmente. Isso ajuda a manter o código do teste limpo (focar no Act e Assert) e evita duplicação de código ao preparar dados de teste. Em DS, criar dados sintéticos representativos é uma habilidade valiosa: construa manualmente pequenos dataframes ou arrays que cubram casos importantes para passar às funções em teste.

Por exemplo, ao testar uma função de agregação por grupo, monte um dataframe de 5-10 linhas com grupos e valores variados, incluindo casos extremos (grupo com um elemento só, grupo com valor nulo etc.). Dados pequenos permitem calcular o resultado esperado “na mão” facilmente, para usar na asserção. Muitas vezes, vale a pena montar esses fixtures cobrindo:

- Valores normais;
- Valores de borda (0, negativos, nulos, strings vazias etc.);
- Formatos alternativos (ordem trocada de colunas, input não ordenado, etc.). Isso garante que a função seja testada além do cenário trivial.

Lembre-se de limpar efeitos colaterais também: se o teste criar um arquivo temporário ou inserir algo em um banco, deve remover depois (Pytest tem fixtures com `teardown` para isso). Por exemplo, o teste de uma função que salva um modelo em disco deve remover o arquivo no final para não poluir o ambiente de testes ou usar uma pasta temporária.

4. Evite dependência entre testes: cada teste deve poder rodar independentemente, em qualquer ordem. Não use resultados de um teste como entrada de outro. Se precisar de uma configuração comum (ex.: carregar um

pequeno dataset), use fixtures compartilhados em vez de fazer um teste chamar outro.

Isso é importante para isolamento – um bug não deve causar cascata de falhas confusas. Também permite rodar testes em paralelo (usando por exemplo `pytest -n auto` para distribuir testes entre núcleos de CPU). Em projetos de DS com muitos testes, isso pode acelerar a validação do código sem comprometer a integridade.

5. Nomeie claramente os testes e indique o caso de uso: bons nomes ajudam futuros(as) desenvolvedores(as) (ou você mesmo(a), meses depois) a entender o objetivo. Por exemplo, `test_filtira_negativos_remove_apenas_negativos` já dá ideia do que está checando. Se o framework suportar, use parâmetros no nome para distinguir cenários – o Pytest por padrão mostra os parâmetros que fizeram um teste paramétrico falhar, ajudando a localizar qual caso quebrou. Adicionar docstrings ou comentários dentro do teste também pode esclarecer a intenção, principalmente para casos não triviais.

6. Aproveite parametrização e markings do Pytest: já demonstramos a vantagem de parametrizar testes para cobrir múltiplos inputs sem repetição. Além disso, o Pytest permite marcar testes com categorias – por exemplo, `@pytest.mark.slow` para testes lentos. Você pode configurá-lo para pular testes marcados como slow na rotina diária e executá-los apenas em builds noturnos ou sob demanda.

Isso é útil em DS: testes que envolvem treinar um modelo por cinco minutos podem ser marcados como lentos e rodados só eventualmente, enquanto testes rápidos rodam sempre. Outro exemplo: `@pytest.mark.skipif` pode pular testes se certas dependências não estão disponíveis (ex.: pular testes de GPU se não há GPU no ambiente de CI). Use essas funcionalidades para manter a suíte eficiente e relevante.

7. Utilize mocking para isolamento de dependências externas: mocking significa simular comportamentos de componentes que o código em teste interage, mas que não são o foco do teste. Em ciência de dados, cenários típicos:

- Mockar I/O de rede ou banco de dados: se sua função baixa dados de uma API, nos testes você não quer depender da API real (que pode estar

fora do ar ou retornar dados diferentes). Use um mock que imita a resposta da API com dados fixos. Bibliotecas como `unittest.mock` ou `requests-mock` permitem interceptar chamadas HTTP e fornecer respostas pré-definidas. Assim você testa apenas como o código processa os dados da API, sob condições controladas.

- Mockar tempo ou aleatoriedade: por exemplo, para testar uma função que pega a data atual, fixe um `datetime` fake.
- Mockar funções pesadas: se na integração você quer incluir um módulo de treinamento de modelo, mas não quer que ele treine de fato (para não gastar tempo), você pode substituir a chamada de `treinar_modelo` por uma função fictícia que retorna um modelo pré-criado ou resultados fixos. Cuidado: deve-se usar mocking criteriosamente – se você simula muita coisa, o teste pode perder o sentido.

Uma crítica em ML é que “não se deve mockar modelos em testes de unidade”, porque às vezes você quer realmente rodar partes do modelo para ter certeza que ele funciona end-to-end. Uma recomendação é: em unit tests, isole e mocke dependências sim, mas em testes de integração procure usar o máximo real possível (talvez usando versões reduzidas, como treinar modelo por 1 época ou com menos dados, em vez de mockar completamente o treino). Assim, você valida a integração de verdade, pegando problemas entre componentes, não só componentes isolados.

8. Teste também cases de erro e comportamentos esperados em condições inválidas: um software robusto lida graciosamente com inputs ruins – seu código de DS deveria, por exemplo, lançar uma exceção clara se receber dados malformados em vez de falhar silenciosamente ou produzir uma saída errada. Escreva testes que passem inputs inválidos (ex.: `None` onde não deveria, tipo de dado errado, coluna faltando no `DataFrame`) e verifique se o código reage corretamente – seja levantando uma exceção do tipo certo com mensagem explicativa, seja retornando um valor padrão/documentado. Isso garante que você não só valida o “caminho feliz”, mas também que erros são tratados conforme o contrato da função. Por exemplo:

```
import pytest
def test_calcula_media_input_invalido():
    with pytest.raises(TypeError):
        calcula_media("texto em vez de lista") # deve disparar TypeError
```

Código-fonte 4 – Exemplo de código para simulação de testes inválidos em Python
Fonte: Elaborado pelo autor (2025)

Se futuramente alguém mudar `calcula_media` e fizer com que, em vez de `TypeError`, ela retorne 0 silenciosamente para um string, esse teste vai falhar indicando mudança indevida no contrato.

9. Monitore cobertura de testes, mas use com sabedoria: ferramentas como `coverage.py` medem quantos % das linhas (ou ramos) do código são exercitados pelos testes. É uma métrica útil para encontrar partes não testadas do código que merecem atenção. Em projetos produzidos, as equipes frequentemente definem um percentual mínimo de cobertura (e.g. 80%) que deve ser mantido.

No entanto, não viciie em perseguir 100% sem critério – mais vale cobertura qualitativa dos pontos críticos do que números inflados por testes triviais. Por exemplo, testar getters e setters só para cobrir linhas não agrega valor; foque nas lógicas de decisão, cálculos e integrações complexas.

Uma técnica: examine relatórios de cobertura para identificar áreas desprotegidas (marcadas em vermelho) e avalie se são importantes. Talvez uma função auxiliar simples não precise de teste dedicado, mas um caminho não usual de uma função complexa sim. Ferramentas de cobertura também podem ajudar a detectar código morto (nunca executado) – nesse caso, ou você remove o código ou escreve teste se ele for relevante mas não exercido em nenhum cenário.

10. Pense em desempenho dos testes (dilema: amostras pequenas vs. realistas): um desafio em DS é que funções às vezes só fazem sentido com muitos dados. Treinar um modelo com 5 linhas não reflete a realidade. Porém, testes precisam ser rápidos para serem executados com frequência. A prática comum é usar amostras reduzidas de dados nos testes – suficientes para testar a lógica, mas pequenas o bastante para execução ágil. Exemplo: se a pipeline real treina um modelo em 1 milhão de exemplos, um teste pode treinar o mesmo modelo em 100 exemplos sintéticos; ele não avaliará a performance preditiva real, mas validará que o código roda sem erro e produz uma saída estruturada (um modelo válido).

Em certos casos, pode-se simular dados com propriedades específicas para estressar o algoritmo em pouco tempo. Por exemplo, para testar um algoritmo de clustering, você pode gerar pontos aleatórios já separados em 3 grupos bem distintos e verificar se o algoritmo retorna 3 clusters com baixa variação intracluster – isso com, digamos, 300 pontos, em vez de milhões.

Para componentes críticos de performance, considere escrever testes de desempenho separados (não tanto para passar/falhar, mas para monitorar e alertar se o tempo piorar além de certo limite). Exemplo: um teste que processa 100k linhas e falha se demorar mais que 2 segundos pode pegar regressões de complexidade (alguém inadvertidamente trocou um algoritmo linear por um quadrático, por exemplo). Entretanto, evitar flakiness aqui é difícil, pois o tempo de execução pode variar – então tais testes podem ser marcados como benchmarking, não rodando em cada commit, mas guardando histórico de tempos em cada versão.

11. Utilize conceitos matemáticos e algoritmos conhecidos para validar resultados: quando possível, compare a saída do seu código com resultados calculados por outro método ou conhecido analiticamente. Por exemplo, se você escreve um algoritmo para calcular medianas móveis, pode testar contra uma implementação base line (talvez menos eficiente) – ex.: usar Pandas ou até mesmo uma fórmula; assim se seu algoritmo otimizado tiver bug, o teste pega pela divergência.

Para um modelo de regressão linear custom, você pode comparar com os coeficientes produzidos pelo `sklearn.LinearRegression` no mesmo conjunto de dados. Em alguns casos, é possível verificar propriedades teóricas: um teste para um algoritmo de ordenação pode embaralhar uma lista e depois de ordenar verificar que:

- A lista resultante está ordenada ($\forall i, \text{lista}[i] \leq \text{lista}[i+1]$),
- A lista resultante é permutação da original (contém os mesmos elementos, possivelmente usando uma comparação de multi-conjuntos ou somatório). Dessa forma não precisamos saber exatamente a saída, mas sim que ela cumpre condições formais de “lista ordenada”. Testes baseados em propriedades têm potencial de cobrir inúmeros casos e muitas vezes revelam erros em cantos não testados por exemplos fixos.

12. Não negligencie testes para código de infraestrutura e utilitários: projetos de ciência de dados frequentemente incluem scripts utilitários (por exemplo, funções para carregar dados, fazer log, monitorar memória etc.) e configuração de pipelines (arquivos YAML/JSON, código de orquestração). Sempre que possível, automatize também verificações disso.

Ex.: se há um arquivo de configuração com parâmetros de modelo, escreva um pequeno teste que verifica que todos os campos obrigatórios estão presentes e com tipos válidos (pode ser tão simples quanto carregar o JSON e validar chaves).

Se há um script de ETL escrito em SQL ou Spark, talvez valha a pena rodá-lo em um banco de teste/spark local com dados fictícios e confirmar contagens de linhas ou soma de valores como esperado após transformações. Isso evita surpresas quando o pipeline rodar “de verdade” na produção.

13. Incorpore os testes ao fluxo de desenvolvimento (Integração Contínua): por fim, uma prática fundamental de engenharia de software que se aplica igualmente a projetos de DS: automatize a execução dos testes no processo de desenvolvimento e deployment. Configure ferramentas de CI (como GitHub Actions, GitLab CI, Azure DevOps etc.) para rodar a suíte de testes a cada push ou pull request.

Assim, se um colaborador ou colaboradora faz uma mudança que quebra algo, o sistema avisa antes de integrar o código. Isso impõe disciplina e mantém a qualidade no crescimento do projeto. Muitos problemas em ciência de dados acontecem meses depois, quando alguém reaproveita um código antigo sem perceber que uma mudança posterior alterou seu comportamento – a CI com testes impediria isso, porque qualquer alteração que mudasse o comportamento deveria ser refletida ou justificada nos testes.

Além disso, quando for colocar um modelo em produção ou agendar uma pipeline regularmente, tenha os testes rodando como “gate”: o deploy só acontece se todos os testes passarem, assegurando que a versão nova está ok.

Integrar com coverage na CI também ajuda a não deixar a cobertura cair sem perceber. E lembre-se das recomendações para a escrita desta aula: sempre verifique se os links fornecidos nas referências existem (nossos testes neste texto de certa forma são as citações verificadas!) e de que um conteúdo muito importante

deve ser trazido para o material em vez de apenas linkado – por isso trazemos tantas práticas aqui em detalhe.

Resumindo as melhores práticas: teste cedo, teste com frequência, cubra casos normais e de erro e mantenha seus testes tão bem cuidados quanto o próprio código de produção. Testes não são “acessórios”, mas sim parte integral do código – devem ser claros, confiáveis e atualizados quando o código muda. Em troca, eles lhe darão a confiança para evoluir o projeto rapidamente e a tranquilidade de que os resultados que você obtém de seus pipelines e modelos estão corretos e estáveis.

Considerações sobre Recursos: Memória, CPU e Ambiente de Execução nos Testes

Projetos de ciência de dados podem exigir recursos intensivos – processar big data, treinar redes neurais profundas etc. Como garantir que nossos testes não só passem logicamente, mas também nos ajudem a monitorar e controlar o uso de recursos? Vamos discutir alguns pontos.

Uso de Memória: Em Python, especialmente com bibliotecas como Pandas e numpy, é fácil consumir muita RAM com grandes datasets. Enquanto nos testes geralmente usamos dados pequenos (então não há problema de memória), podemos escrever testes específicos para monitorar **leaks de memória**. Por exemplo, se implementamos uma classe com um método que deve liberar grandes estruturas após uso, um teste pode chamar esse método repetidamente em um loop e verificar via o módulo tracemalloc se a memória alocada não aumenta a cada iteração. Outra técnica é usar fixtures de setup/teardown para medir a memória antes e depois de certas chamadas. Em casos de pipelines de streaming, podemos simular um fluxo contínuo e garantir que o uso de memória estabiliza (indicando que o coletor de lixo está funcionando e não há acúmulo indefinido). Ferramentas externas como memory_profiler podem ser integradas para checar picos de memória em certas funções.

Uso de CPU/GPU: testes de desempenho já comentados cobrem CPU em termos de tempo. Às vezes, pode ser interessante monitorar se algoritmos paralelizados estão de fato usando múltiplos núcleos ou GPUs. Por exemplo, se temos uma implementação numba ou multiprocessing, podemos cronometrar com

tamanhos crescentes de input para ver se o tempo escala sub-linearmente (esperado se paralelizou).

Para GPU, podemos inserir hooks (por ex., se usando PyTorch, verificar que a device de um tensor resultante realmente é “cuda:0” e não “cpu” – isso garante que a operação ocorreu na GPU como pretendido). Em cenários de desempenho crítico, testes podem ajudar a identificar regressões – e se bem construídos, até alertar para mudança de complexidade algorítmica como mencionado ($O(n)$ virando $O(n^2)$ acidentalmente).

Ambiente (compatibilidade e dependências): projetos de DS dependem muito de bibliotecas (NumPy, Pandas, Scikit-learn, etc.). Problemas de versão ou ambientes diferentes (Windows vs Linux, diferentes versões de Python) podem introduzir comportamentos distintos. Uma suite robusta de testes acaba funcionando também como verificação de portabilidade: ao rodar os testes em múltiplos ambientes (é comum configurar a CI para rodar contra, digamos, Python 3.9, 3.10 e 3.11; ou contra Spark 3.0 e 3.1; etc.), você detecta se algo falha somente em um certo contexto.

Por exemplo, um teste que passa em Linux pode falhar no Windows por causa de path de arquivo: isso expõe a necessidade de lidar com separadores de diretório no código. Assim, é válido incluir uma matriz de teste para ambientes se seu projeto pretende ser multiplataforma ou funcionar com múltiplas versões de libs. Tenha cuidado apenas de não abusar – rodar toda suíte para 10 combinações diferentes pode multiplicar o tempo total. Foque nas combinações críticas (talvez a versão mínima suportada e a mais nova das dependências) ou teste condicionalmente o que varia.

Testes distribuídos e concorrência: se sua pipeline envolve processamento distribuído (por exemplo, jobs Spark, Flink, Dask) ou pipelines assíncronas, escrever testes traz desafios extras, mas possíveis de contornar. Alguns pontos:

- Para frameworks como Spark, pode-se usar um Spark local (modo local[*], que executa em threads locais) nos testes de integração. Isso permite rodar código Spark real em memória, embora limitando paralelismo. Testes podem criar um SparkSession com opção local e usar dados pequenos paralelizáveis. Assim você verifica a lógica de transformations e

actions Spark. Cuidados: a saída de uma operação paralela às vezes não tem ordem definida – seus testes devem levar isso em conta (por exemplo, ordenar a saída ou compará-la como conjuntos). Além disso, liberar o contexto Spark após teste (para não interferir em outros).

- Para sistemas de fila de mensagens ou streaming, considere usar ambientes embarcados. Ex.: se há consumo de um Kafka, nos testes use o Testcontainers para subir um Kafka temporário e injetar mensagens de teste e então verificar se o output foi produzido corretamente. Parece complexo, mas há bibliotecas de teste justamente para simular esses componentes. No mínimo, se a lógica de consumo/produção estiver bem abstrata, você pode mockar a interface do streaming e testar a lógica de negócio isoladamente.
- Concorrência e thread-safety: se seu código lança threads ou processos (por exemplo, uma função de predição que paraleliza várias chamadas de modelo), valem testes para condições de corrida. Isso pode envolver rodar a função muitas vezes em paralelo e verificar resultados consistentes ou ausência de condições de erro (pegar exceções ou resultados incorretos que poderiam indicar problemas). Aqui, ferramentas como pytest-xdist (que executa testes em paralelo) podem revelar problemas se testes não forem escritos com isolamento – se um recurso global não protegido for acessado concorrentemente, testes paralelos podem falhar de forma não determinística. Portanto, corrigir testes para rodar em paralelo acaba também forçando a tornar o código thread-safe.
- Em Python, devido ao GIL (antes da versão 3.14), a concorrência de CPU não é real multi-threading, mas multi-processing ou I/O-bound multi-threading. Testar multi-process (via multiprocessing or joblib Parallel) requer possivelmente estrutura diferente – às vezes, não dá para usar certos mocks dentro de processos filhos. Nesses casos, testes podem focar no resultado final do paralelismo (ex.: calculou todos os chunks e agregou corretamente), em vez de tentar inspecionar cada sub-processo.

Documentação e manutenibilidade dos testes: lembre-se que testes também servem como documentação viva do comportamento esperado. Então

mantê-los atualizados quando requisitos mudam é importante. Se um comportamento muda legitimamente (por exemplo, decidimos que `calcula_media` deve retornar `None` em vez de `0` para lista vazia), então primeiro escrevemos ou alteramos o teste para refletir o novo comportamento e em seguida ajustamos o código – seguindo o TDD mesmo para mudança de requisito. Os testes falharão inicialmente (indicando que o código antigo não satisfaz o novo requisito) e depois passarão quando o código for adequado. Isso garante que você não esqueça de nenhum ponto em que aquele comportamento importava.

Um cuidado é não tornar os testes frágeis em relação a detalhes de implementação que não importam para o contrato externo do código. Por exemplo, se a ordem de colunas em um `DataFrame` de saída não foi especificada como garantida, talvez seu teste de integração devesse comparar os valores independentemente da ordem de colunas, para não quebrar caso o código mude a ordem sem afetar a correção lógica. Foque em o que o código deve fazer, não como. Testes muito acoplados à implementação impedem melhorias; ex.: testar que uma função usa internamente `np.mean` pode ser exagerado – melhor testar que o resultado calculado é correto, não importa como.

Casos Especiais: Testando Modelos de Machine Learning e Pipelines de Big Data

Vale destacar algumas situações específicas em DS:

Testando modelos de ML: como testar que um modelo de Machine Learning está “funcionando”? Afinal, seu objetivo é generalizar bem para dados futuros, algo que foge a testes unitários determinísticos. Ainda assim, há vários aspectos testáveis:

- **Sanidade no treinamento:** um truque conhecido é testar se o modelo pode overfit um conjunto muito pequeno de dados. Por exemplo, pega-se 10 amostras de treino e usa-se o próprio modelo para prever nelas – espera-se (para modelos suficientemente flexíveis) que ele consiga obter erro próximo de zero nesses mesmos pontos após treinamento.

Se não consegue, pode haver um bug na função de treino (p. ex., os pesos não estão sendo atualizados corretamente). Isso é TDD no sentido de comportamento: “modelo treinado em dados minúsculos deve praticamente os memorizar” – escrevemos esse teste, rodamos o treinamento por algumas épocas e validamos que a perda cai abaixo de um limiar ou a acurácia chega a 100% no set minúsculo.

Esse teste garante que a pipeline de treino está ajustando parâmetros como esperado. Exemplo: testar um classificador simples rodando em 5 exemplos e verificar que ele consegue classificar perfeitamente esses 5 (mesmo que não signifique nada para o mundo real, dá confiança de que o gradiente está certo etc.). Esse tipo de teste foi reportado por engenheiros do Google como extremamente útil para pegar erros de implementação de redes neurais complexas – se a rede não consegue overfit um único lote, tem algo muito errado.

- **Comparação com implementação de referência:** como citado, se você escrever seu próprio algoritmo de regressão linear ou árvore de decisão, compare os resultados com scikit-learn ou outra biblioteca confiável nos mesmos dados. Os coeficientes ou a saída exata podem não ser bit a bit idênticos (podem haver diferenças numéricas ou de critérios de parada), mas você pode validar que ou são quase iguais, ou que sua implementação atinge precisão similar em algum conjunto de teste. Isso análogo a testar uma nova solução contra uma conhecida (mesmo que mais lenta).
- **Testes de invariantes em modelos:** alguns modelos têm propriedades que podemos checar. Ex.: redes neurais com função de ativação sigmoide devem produzir saída no intervalo (0,1). Então após treinar uma rede, passe alguns exemplos e teste que $0 \leq y_{\text{pred}} \leq 1$ para todos – a violação disso indica bug (por exemplo, esquecer de aplicar ativação na última camada).

Outros invariantes: uma SVM linear sem regularização muito alta pode ficar com coeficientes muito grandes – mas não deve retornar NaN; se isso ocorrer, há instabilidade. Em modelos de clustering como K-means: após

rodar, valide que todos pontos de teste estão atribuídos a algum cluster (nenhum ficou sem etiqueta) e que o número de clusters retornados é o solicitado. Pequenos checks assim pegam erros de lógica.

- **Testar persistência de modelo:** se você salva e carrega modelos, teste que um modelo salvo e recarregado produz as mesmas previsões que antes (dado o mesmo input). Isso garante que a serialização está funcionando e não perdendo informação. Por exemplo, com scikit-learn: treine um RandomForest no test, salve com joblib.dump, limpe da memória, recarregue e compare previsões em um conjunto de amostra com o original. Devem bater bit a bit.
- **Testes de vieses e equidade:** para modelos éticos, pode ser útil incluir testes que verifiquem se certas propriedades se mantêm. Ex.: rodar um modelo de classificação e verificar que a taxa de falsos positivos para dois subgrupos demográficos não difere além de X%. Isso entra mais em validação de modelo do que em teste de código, mas algumas equipes automatizam testes de fairness nos pipelines – se o modelo novo falha nesses critérios, nem é aprovado para produção. É um cenário avançado, mesclando QA de software com validação estatística.

Testando pipelines de big data/ETL: em pipelines robustas (com Spark, Hadoop etc.), muitas etapas não são facilmente “unit testáveis” sem contexto (por ex., uma transformation Spark depende de um SparkContext). Neste caso, adota-se frequentemente testes de integração em miniatura: instanciar o motor de processamento em modo local ou test (como discutido) e rodar toda a sequência com poucos dados. Isso já cobrirá a maioria das transformações.

Para simular falhas em fases, às vezes injeta-se dados com características problemáticas: ex. um arquivo CSV com linha corrompida para garantir que o pipeline pula ou relata erro adequadamente. Pode-se ainda usar ferramentas de monkeypatch para simular exceções em pontos específicos e ver se o pipeline inteiro responde certo (ex.: banco de dados indisponível – se sua pipeline deve tentar 3 vezes e depois registrar erro, você pode mockar a conexão para lançar exceção e verificar no log de saída que 3 tentativas foram feitas e o erro final foi logado).

Outra técnica interessante: testes de fumaça (smoke tests). São testes bem simples que checam se componentes principais podem ser carregados e executados sem crash. Por exemplo, um smoke test de pipeline ETL pode simplesmente executar a função principal em modo de teste (com uma opção `dry_run=True`) que faz todos os passos menos gravar resultados e terminar. Se isso roda sem exceções, pelo menos sabemos que todas as dependências estão resolvidas, caminhos de arquivos existem, credenciais ok etc. Smoke tests geralmente não validam resultados em detalhe, mas pegam problemas de ambiente e configurações faltantes.

Por fim, lembre-se que testar é investir em qualidade. Pode parecer muito trabalho no começo, mas a longo prazo economiza tempo de depuração e confere confiabilidade aos seus entregáveis de ciência de dados. Muitas falhas custosas poderiam ter sido evitadas se testes automatizados tivessem sido implementados – na próxima seção, veremos alguns casos reais de sucesso e insucesso relacionados a isso.

MERCADO, CASES E TENDÊNCIAS

Knight Capital (2012) – Bug de US\$ 440 milhões

Em agosto de 2012, um erro de software quase levou a Knight Capital à falência em 45 minutos, causando um prejuízo de US\$ 440 milhões após execuções erradas por código legado. A falta de testes adequados, como regressão e end-to-end, permitiu o problema. O caso virou referência sobre a importância de testes abrangentes e protocolos seguros no setor financeiro, levando a práticas como deploy gradativo e simulações antes de mudanças.

O livro “Test-Driven Development with Python” (Harry Percival) embora focado em web, inspirou muitos a usarem TDD; sua terceira edição (2023) discute inclusive TDD na época da IA e ressalta que a prática continua relevante para produzir “código limpo que funciona”.

Blogs como o de **Eugenia Yan**, ex-engenheiro da Amazon, trazem artigos como “Don’t Mock ML Models in Unit Tests” e “[Writing Robust Tests for Data & ML Pipelines](#)”, cheios de dicas práticas e exemplos de código Pytest para pipelines de recomendação etc. (parte das referências usadas neste texto).

Ferramentas open source estão ganhando adoção: além de Great Expectations, há o Deepchecks (focado em testes para modelos de ML – inclusive testes de performance e de fairness) e o Checklist (da Microsoft Research, para testar comportamentos de modelos de NLP).

O QUE VOCÊ VIU NESTA AULA?

Exploramos o universo de Testes Automatizados aplicados à Engenharia de Software para Cientistas de Dados. Iniciamos com uma introdução conceitual sobre a importância de testes em pipelines e modelos – prevenindo regressões e trazendo confiabilidade.

Passamos por uma sessão prática em que implementamos testes unitários e de integração em Python com Pytest, demonstrando na prática os benefícios do TDD e da verificação passo-a-passo de uma solução.

Em seguida, nos aprofundamos teoricamente, cobrindo desde tipos de teste (unitário, integração, end-to-end) e TDD até melhores práticas, aspectos matemáticos, controle de recursos e peculiaridades de testar aprendizado de máquina e big data – sempre com exemplos concretos e fontes confiáveis para embasar.

Por fim, analisamos casos reais do mercado, extraíndo lições de grandes fracassos evitáveis e sucessos obtidos graças a uma cultura de testes. Com isso, consolidamos a visão de que testar não é obstáculo, mas sim parte integrante do trabalho do indivíduo cientista/engenheiro de dados moderno.

Ao levar adiante esses conhecimentos, você estará mais preparado(a) para entregar projetos de data science com qualidade de software de ponta, ganhando confiança de colegas, gestores(as) e usuários nos produtos de dados que desenvolver. Boa prática e bons testes!

REFERÊNCIAS

CHOREV, S. **Top 10 ML Model Failures You Should Know About**. 2022. Disponível em: <https://www.deepchecks.com/top-10-ml-model-failures-you-should-know-about/>. Acesso em: 12 dez. 2025.

GREAT EXPECTATIONS. **How Calm uses GX to create data quality alerts and avert critical data issues in Airflow DAGs**. 2024. Disponível em: <https://greatexpectations.io/case-studies/how-calm-uses-gx-to-create-data-quality-alerts-and-avert-critical-data/>. Acesso em: 12 dez. 2025.

NELSON, C. **Software Engineering for Data Scientists**. Sebastopol: O'Reilly Media, 2024. Disponível em: <https://unidel.edu.ng/focelibrary/books/Software%20Engineering%20for%20Data%20Scientists%20%28Catherine%20Nelson%29%20%28Z-Library%29.pdf/>. Acesso em: 12 dez. 2025.

PERCIVAL, H. **Test-Driven Development with Python**. 3. ed. Sebastopol: O'Reilly Media, 2025.

YAN, E. **Writing Robust Tests for Data & Machine Learning Pipelines**. 2023. Disponível em: <https://eugeneyan.com/writing/testing-pipelines/>. Acesso em: 12 dez. 2025.

PALAVRAS-CHAVE

Testes Automatizados. Engenharia de Software para Ciência de Dados. Test-Driven Development (TDD).

UNIVERSIDADE

EMAP